



Principios SOLID

Los principios **SOLID**, introducidos por **Robert C. Martin** (conocido como "Uncle Bob"), representan el estándar de oro en la ingeniería de software para combatir la rigidez y la fragilidad del código. No se trata simplemente de reglas de escritura, sino de una mentalidad de diseño orientada a la supervivencia del software en el tiempo.

El Combate contra la Putrefacción del Software

Todo sistema exitoso tiende a cambiar. Sin una base sólida, el software sufre de lo que Martin define como "putrefacción":

- **Rigidez:** La tendencia del software a ser difícil de cambiar, donde un pequeño ajuste requiere una cascada de modificaciones en otros módulos.
- **Fragilidad:** Cuando realizamos un cambio, el sistema se rompe en lugares que no tienen relación directa con el área modificada.
- **Inmovilidad:** La incapacidad de reutilizar partes del software en otros proyectos porque los módulos están excesivamente entrelazados.

El Poder de las Interfaces y la Abstracción en Java

En el ecosistema de Java, estos principios se materializan mediante el uso estratégico de **Interfaces** y **Polimorfismo**. El objetivo es que el código de alto nivel (la lógica de negocio) no dependa de los detalles de bajo nivel (como una base de datos específica o una librería de terceros).

Impacto en la Mantenibilidad y el Costo del Cambio

La aplicación de estos principios reduce drásticamente la **Deuda Técnica**. Al diseñar componentes que están "desacoplados", el **Costo del Cambio** deja de crecer de forma exponencial. Esto permite que el sistema sea:

- **Extensible:** Podemos agregar nuevas funcionalidades añadiendo código nuevo, en lugar de modificar constantemente el código existente que ya funciona.
- **Testeable:** Al separar responsabilidades, las pruebas unitarias se vuelven sencillas de implementar, ya que podemos aislar cada componente.



En conclusión, los principios de Robert C. Martin buscan que el código sea lo suficientemente flexible como para aceptar los cambios inevitables del negocio sin que la estructura interna colapse bajo el peso de su propia complejidad.

Son **5 principios para diseñar software mantenible**.

- **S - SRP** (Single Responsibility Principle): Una clase debe tener **una sola razón para cambiar** (una clase debería hacer solo una cosa).

Ejemplo

✗ Mala práctica:

```
public class UsuarioService {  
    public void guardarUsuario() { }  
    public void enviarEmail() { }  
}
```

✓ Buena práctica:

```
public class UsuarioService {  
    public void guardarUsuario() { }  
}
```

```
public class EmailService {  
    public void enviarEmail() { }  
}
```

- **O - OCP** (Open/Closed Principle): El software debe estar **abierto para extensión pero cerrado para modificación** (agregar nuevas funcionalidades, sin tocar código que ya funciona -> abstracción con interfaces).

Ejemplo

✗ Mala práctica:

```
public double calcular(String tipo) {  
    if(tipo.equals("A")) return 10;  
    if(tipo.equals("B")) return 20;  
    return 0;  
}
```



✓ Buena práctica:

```
interface Tipo {  
    double calcular();  
}  
class TipoA implements Tipo {  
    public double calcular() { return 10; }  
}  
class TipoB implements Tipo {  
    public double calcular() { return 20; }  
}
```

- **L - LSP** (Liskov Substitution Principle): Las subclases deben poder reemplazar a la clase padre sin romper el sistema.

Ejemplo

✗ Mala práctica:

```
class Ave {  
    void volar() { }  
}
```

```
class Pinguino extends Ave {  
    void volar() {  
        throw new UnsupportedOperationException();  
    }  
}
```

✓ Buena práctica:

```
interface Ave { }
```

```
interface AveVoladora {  
    void volar();  
}
```

```
class Paloma implements Ave, AveVoladora {  
    public void volar() { }  
}
```



- **I - ISP** (Interface Segregation Principle): Ningún cliente debería verse obligado a depender de métodos que no utiliza. Este principio propone dividir interfaces grandes en interfaces más pequeñas y específicas, de modo que cada clase implemente solo las operaciones que realmente necesita.

Ejemplo

✗ Mala práctica:

```
interface Trabajador {  
    void trabajar();  
    void comer();  
}
```

✓ Buena práctica:

```
interface Trabajar {  
    void trabajar();  
}
```

```
interface Comer {  
    void comer();  
}
```

- **D - DIP** (Dependency Inversion Principle): Depender de **abstracciones**, no de implementaciones. Es decir que el código no dependa directamente de **clases concretas**, sino de **interfaces o abstracciones**, para reducir el acoplamiento y facilitar cambios en el sistema.

Ejemplo

✗ Mala práctica:

```
class Motor {  
}
```

```
class Auto {  
    private Motor motor = new Motor();  
}
```

✓ Buena práctica:
interface Motor {
}

```
class MotorElectrico implements Motor {  
}
```

```
class Auto {  
    private Motor motor;  
  
    public Auto(Motor motor) {  
        this.motor = motor;  
    }  
}
```